

Model Registry & CI/CD Pipeline (DAG): AI-Powered Apple Leaf Specialist

- **Name:** Aktham Almomani
- **Group:** 7

Introduction

This notebook builds a CI-style SageMaker pipeline that trains, evaluates, gates on quality, and registers an Apple leaf classifier in the SageMaker Model Registry, with approval and deployment.

Here're the main steps in this notebooks:

- Define a reusable pipeline that produces versioned model packages.
- Enforce a validation-accuracy threshold before registration.
- Persist evaluation metrics alongside each registered version.
- Finally, approve and deploy a chosen package to an endpoint.

Setup and Parameters

Let's first start by initializing SageMaker session, role, region, and declarative pipeline parameters such as S3 paths, package group, and accuracy threshold. Introduce caching for faster re-runs.

```
# imports:
import sagemaker, boto3, json
from sagemaker.workflow.pipeline import Pipeline
from sagemaker.workflow.parameters import ParameterString,
ParameterFloat
from sagemaker.workflow.steps import TrainingStep, ProcessingStep,
CacheConfig
from sagemaker.processing import ScriptProcessor, ProcessingInput,
ProcessingOutput
from sagemaker.workflow.conditions import import
ConditionGreaterThanOrEqualTo
from sagemaker.workflow.condition_step import ConditionStep
from sagemaker.workflow.fail_step import FailStep
from sagemaker.pytorch import PyTorch
from sagemaker.workflow.model_step import ModelStep
from sagemaker.workflow.properties import PropertyFile
from sagemaker.workflow.functions import JsonGet
from sagemaker import image_uris

sess    = sagemaker.Session()
role    = sagemaker.get_execution_role()
```

```

region = sess.boto_region_name

# pipeline params:
bucket_p = ParameterString("Bucket",
default_value=sess.default_bucket())
prefix_p = ParameterString("Prefix", default_value="apple/pipeline")
train_s3_p = ParameterString("TrainS3",
default_value=f"s3://{sess.default_bucket()}/apple/train.zip")
val_s3_p = ParameterString("ValS3",
default_value=f"s3://{sess.default_bucket()}/apple/val.zip")
acc_thr_p = ParameterFloat("MinValAcc", default_value=0.98)
pkg_group_p = ParameterString("ModelPackageGroup",
default_value="apple-leaf-registry")

cache = CacheConfig(enable_caching=True, expire_after="30d")

sagemaker.config INFO - Not applying SDK defaults from location:
/etc/xdg/sagemaker/config.yaml
sagemaker.config INFO - Not applying SDK defaults from location:
/home/sagemaker-user/.config/sagemaker/config.yaml

```

Training Step

Here let's create a PyTorch estimator pointing to `src/train.py`, set hyperparameters, and define a TrainingStep that consumes train and validation datasets from S3.

```

# train:
est = PyTorch(
    entry_point="train.py",
    source_dir="src",
    framework_version="2.3",
    py_version="py311",
    role=role,
    instance_type="ml.m5.xlarge",
    instance_count=1,
    hyperparameters={"epochs":5, "img_size":256, "bs":32, "lr":1e-3},
    sagemaker_session=sess,
)

train_step = TrainingStep(
    name="TrainModel",
    estimator=est,
    inputs={"train": train_s3_p, "validation": val_s3_p},
    cache_config=cache,
)

/opt/conda/lib/python3.12/site-packages/sagemaker/workflow/
steps.py:485: UserWarning: Profiling is enabled on the provided
estimator. The default profiler rule includes a timestamp which will

```

change each time the pipeline is upserted, causing cache misses. If profiling is not needed, set `disable_profiler` to `True` on the estimator.

```
warnings.warn(msg)
```

```
# Get a PyTorch training image for CPU:
```

```
eval_image = image_uris.retrieve(  
    framework="pytorch",  
    region=region,  
    version="2.3",  
    image_scope="training",  
    py_version="py311",  
    instance_type="ml.m5.xlarge",  
)
```

```
proc = ScriptProcessor(  
    image_uri=eval_image,  
    role=role,  
    instance_type="ml.m5.xlarge",  
    instance_count=1,  
    sagemaker_session=sess,  
    command=["python3"],  
)
```

Evaluation Step

Run `src/evaluate.py` in a `ScriptProcessor` to score the trained model against validation data and write `metrics.json` (includes `val_accuracy`) for downstream gating and model metrics.

```
# first let's define the PropertyFile:
```

```
metrics_prop = PropertyFile(  
    name="EvalMetrics",  
    output_name="metrics",  
    path="metrics.json"  
)
```

```
# Then let's build the step and attach property_files:
```

```
eval_step = ProcessingStep(  
    name="EvaluateModel",  
    processor=proc,  
    code="src/evaluate.py",  
    inputs=[  
        ProcessingInput(  
  
source=train_step.properties.ModelArtifacts.S3ModelArtifacts,  
        destination="/opt/ml/processing/model",  
        input_name="model",  
    ),  
)
```

```

        ProcessingInput(
            source=val_s3_p,
            destination="/opt/ml/processing/val",
            input_name="val",
        ),
    ],
    outputs=[
        ProcessingOutput(
            output_name="metrics",
            source="/opt/ml/processing/output"
        )
    ],
    property_files=[metrics_prop],
    cache_config=cache,
)

val_acc_expr = JsonGet(
    step_name=eval_step.name,
    property_file=metrics_prop,
    json_path="val_accuracy"
)

```

Quality Gate

Here let's use a `ConditionStep` to compare `val_accuracy` to `MinValAcc`. On failure, route to a `FailStep` with a clear message; on success, continue to registration.

```

from sagemaker.workflow.functions import Join

# gate: metric vs threshold:
cond = ConditionGreaterThanOrEqualTo(left=val_acc_expr,
right=acc_thr_p)

# build the error message at runtime:
fail_msg = Join(
    on="",
    values=[
        "Validation accuracy below threshold: ",
        val_acc_expr.to_string(), " < ", acc_thr_p.to_string()
    ],
)

fail_step = FailStep(
    name="FailIfLowAccuracy",
    error_message=fail_msg
)

```

Register Model

Here let's register the model into a `ModelPackageGroup`, attach evaluation artifacts as `ModelMetrics`, and let the step optionally repack inference code (`src/inference.py`) for standardized deployment. Status is `PendingManualApproval` by default.

```
try:
    from sagemaker.workflow.model_step import RegisterModel
except ImportError:
    from sagemaker.workflow.step_collections import RegisterModel

from sagemaker.model_metrics import ModelMetrics, MetricsSource

# Metrics wired from our eval ProcessingStep:
metrics_s3_uri =
eval_step.properties.ProcessingOutputConfig.Outputs["metrics"].S3Output.S3Uri
model_metrics = ModelMetrics(
    model_statistics=MetricsSource(s3_uri=metrics_s3_uri,
    content_type="application/json")
)

# Register using the estimator:
register_step = RegisterModel(
    name="RegisterModel",
    estimator=est,
    model_data=train_step.properties.ModelArtifacts.S3ModelArtifacts,
    content_types=["application/json"],
    response_types=["application/json"],
    inference_instances=["ml.m5.large"],
    transform_instances=["ml.m5.large"],
    model_package_group_name=pkg_group_p,
    model_metrics=model_metrics,

    entry_point="inference.py",
    source_dir="src",
)
```

Failure Path

If the quality gate fails, the pipeline routes to a `FailStep` and halts registration. No model package is created, approval is not requested, and deployment is skipped. Use the training and evaluation job artifacts to diagnose issues, adjust data or hyperparameters, and re-run the pipeline.

```
# Fail step:
fail_step = FailStep(
    name="FailIfLowAccuracy",
    error_message="Validation accuracy below threshold."
```

```

)

# Gate condition: pass if val_acc >= threshold:
cond = ConditionGreaterThanOrEqualTo(left=val_acc_expr,
right=acc_thr_p)

# If pass -> register model, else -> fail:
accuracy_gate = ConditionStep(
    name="AccuracyGate",
    conditions=[cond],
    if_steps=[register_step],
    else_steps=[fail_step],
)

from sagemaker.workflow.pipeline import Pipeline

pipe_name = "apple-ci-model-registry"

pipeline = Pipeline(
    name=pipe_name,
    parameters=[bucket_p, prefix_p, train_s3_p, val_s3_p, acc_thr_p,
pkg_group_p],
    steps=[train_step, eval_step, accuracy_gate],
    sagemaker_session=sess,
)

pipeline.upsert(role_arn=role) # create/update the pipeline:

# Launch an execution (tweak params to force pass/fail):
execution = pipeline.start(
    parameters={
        "Bucket": sess.default_bucket(),
        "Prefix": "apple/pipeline",
        "TrainS3": f"s3://{sess.default_bucket()}/apple/train.zip",
        "ValS3": f"s3://{sess.default_bucket()}/apple/val.zip",
        "MinValAcc": 0.98, # gate
threshold
        "ModelPackageGroup": "apple-leaf-registry", # registry
group
    }
)

print("Started execution:", execution.arn)
execution.wait()
print(execution.list_steps())

INFO:sagemaker.image_uris:image_uri is not presented, retrieving
image_uri based on instance_type, framework etc.
INFO:sagemaker.image_uris:image_uri is not presented, retrieving
image_uri based on instance_type, framework etc.

```

WARNING:sagemaker.workflow.utilities:Popping out 'TrainingJobName' from the pipeline definition by default since it will be overridden at pipeline execution time. Please utilize the PipelineDefinitionConfig to persist this field in the pipeline definition if desired.

WARNING:sagemaker.workflow.utilities:Popping out 'ProcessingJobName' from the pipeline definition by default since it will be overridden at pipeline execution time. Please utilize the PipelineDefinitionConfig to persist this field in the pipeline definition if desired.

WARNING:sagemaker.workflow.utilities:Popping out 'TrainingJobName' from the pipeline definition by default since it will be overridden at pipeline execution time. Please utilize the PipelineDefinitionConfig to persist this field in the pipeline definition if desired.

WARNING:sagemaker.estimator:No finished training job found associated with this estimator. Please make sure this estimator is only used for building workflow config

WARNING:sagemaker.workflow._utils:Popping out 'CertifyForMarketplace' from the pipeline definition since it will be overridden in pipeline execution time.

WARNING:sagemaker.workflow.utilities:Popping out 'ModelPackageName' from the pipeline definition by default since it will be overridden at pipeline execution time. Please utilize the PipelineDefinitionConfig to persist this field in the pipeline definition if desired.

Started execution:

arn:aws:sagemaker:us-east-1:533266958221:pipeline/apple-ci-model-registry/execution/44ir52uc6spb

Traceback (most recent call last)

```
in <module>:27

    24 )
    25
    26 print("Started execution:", execution.arn)
> 27 execution.wait()          # stream logs / wait until done
    28 print(execution.list_steps())
    29
```

/opt/conda/lib/python3.12/site-packages/sagemaker/workflow/pipeline.py
:938 in wait |

```

935 |         waiter = botocore.waiter.create_waiter_with_client(
936 |             waiter_id, model,
self.sagemaker_session.sagemaker_client
937 |         )
    938 |         waiter.wait(PipelineExecutionArn=self.arn)
939 |
940 |     def result(self, step_name: str):
941 |         """Retrieves the output of the provided step if it is
a ``@step`` decorated func
|
| /opt/conda/lib/python3.12/site-packages/botocore/waiter.py:55 in
wait
|
52 |     # Waiter.wait method. This is needed to attach a docstring
to the
53 |     # method.
54 |     def wait(self, **kwargs):
    55 |         Waiter.wait(self, **kwargs)
56 |
57 |     wait.__doc__ = WaiterDocstring(
58 |         waiter_name=waiter_name,
|
| /opt/conda/lib/python3.12/site-packages/botocore/waiter.py:374 in
wait
|
371 |         return
372 |         if current_state == 'failure':
373 |             reason = f'Waiter encountered a terminal
failure state: {acceptor.explan
|    374 |             raise WaiterError(

```

```

375 |         |         |         |         |         name=self.name,
376 |         |         |         |         |         reason=reason,
377 |         |         |         |         |         last_response=response,

```

```

WaiterError: Waiter PipelineExecutionComplete failed: Waiter
encountered a terminal failure state: For expression
"PipelineExecutionStatus" we matched expected path: "Failed"

```

```
import sagemaker, os
```

```

sess = sagemaker.Session()
sm    = boto3.client("sagemaker")
region = sess.boto_region_name

```

```
exec_arn = execution.arn
```

```
# Overall status:
```

```

desc = sm.describe_pipeline_execution(PipelineExecutionArn=exec_arn)
print("Pipeline:", desc["PipelineArn"].split("/")[-1])
print("Status   :", desc["PipelineExecutionStatus"])
print("Message  :", desc.get("FailureReason", ""))

```

```

resp = sm.list_pipeline_execution_steps(PipelineExecutionArn=exec_arn)
steps = resp.get("PipelineExecutionSteps", resp.get("Steps", []))

```

```

def job_console_link(arn: str) -> str:
    if not arn: return ""
    if ":processing-job/" in arn:
        return
    f"https://{region}.console.aws.amazon.com/sagemaker/home?
region={region}#/processing-jobs/{arn.split('/')[-1]}"
    if ":training-job/" in arn:
        return
    f"https://{region}.console.aws.amazon.com/sagemaker/home?
region={region}#/jobs/{arn.split('/')[-1]}"
    if ":model-package/" in arn:
        return
    f"https://{region}.console.aws.amazon.com/sagemaker/home?
region={region}#/model-packages/{arn.split('/')[-1]}"
    return ""

```

```

print("\nSteps:")
for st in steps:
    name    = st.get("StepName") or st.get("Name")
    typ     = st.get("StepType") or st.get("Type") or ""

```

```

status = st.get("StepStatus") or st.get("Status") or ""
fr      = st.get("FailureReason", "")
md      = st.get("Metadata", {}) or {}
job_arn = (
    md.get("ProcessingJob", {}).get("Arn")
    or md.get("TrainingJob", {}).get("Arn")
    or md.get("ModelPackage", {}).get("Arn")
)

print(f"- {name:24s} {typ:14s} {status:10s} {fr or ''}")
if job_arn:
    print("  >", job_console_link(job_arn))

```

Pipeline: apple-ci-model-registry

Status : Failed

Message : Step failure: One or multiple steps failed.

Steps:

- FailIfLowAccuracy	Failed	Validation
accuracy below threshold.		
- AccuracyGate	Succeeded	
- EvaluateModel	Succeeded	
> https://us-east-1.console.aws.amazon.com/sagemaker/home?region=us-east-1#/processing-jobs/pipelines-44ir52uc6spb-EvaluateModel-tA9sXZ2lkr		
- TrainModel	Succeeded	
> https://us-east-1.console.aws.amazon.com/sagemaker/home?region=us-east-1#/jobs/pipelines-44ir52uc6spb-TrainModel-SRVhPqZEZg		

Summary Highlights: The pipeline ran end-to-end, produced metrics, and the automated quality gate correctly blocked model registration because accuracy was below the required threshold.

Success Path

If the quality gate passes, the model is registered into the target `ModelPackageGroup` with attached evaluation metrics and inference assets. The new package version is created in `PendingManualApproval` by default, ready for optional approval and promotion to a deployment endpoint.

```

# Launch an execution:
execution = pipeline.start(
    parameters={
        "Bucket": sess.default_bucket(),
        "Prefix": "apple/pipeline",
        "TrainS3": f"s3://{sess.default_bucket()}/apple/train.zip",
        "ValS3":   f"s3://{sess.default_bucket()}/apple/val.zip",
        "MinValAcc": 0.90,                                     # gate
    }
)
threshold: reduce it to pass

```

```

        "ModelPackageGroup": "apple-leaf-registry",          # registry
    group
    }
)

print("Started execution:", execution.arn)
execution.wait()
print(execution.list_steps())

Started execution:
arn:aws:sagemaker:us-east-1:533266958221:pipeline/apple-ci-model-
registry/execution/7piguyat6xjf
[{'StepName': 'RegisterModel-RegisterModel', 'StartTime':
datetime.datetime(2025, 10, 18, 1, 35, 40, 140000, tzinfo=tzlocal()),
'EndTime': datetime.datetime(2025, 10, 18, 1, 35, 41, 218000,
tzinfo=tzlocal()), 'StepStatus': 'Succeeded', 'Metadata':
{'RegisterModel': {'Arn': 'arn:aws:sagemaker:us-east-
1:533266958221:model-package/apple-leaf-registry/1'}}}, {'AttemptCount':
1}, {'StepName': 'RegisterModel-RepackModel', 'StartTime':
datetime.datetime(2025, 10, 18, 1, 33, 6, 188000, tzinfo=tzlocal()),
'EndTime': datetime.datetime(2025, 10, 18, 1, 35, 39, 584000,
tzinfo=tzlocal()), 'StepStatus': 'Succeeded', 'Metadata':
{'TrainingJob': {'Arn': 'arn:aws:sagemaker:us-east-
1:533266958221:training-job/pipelines-7piguyat6xjf-RegisterModel-
Repack-0CnCAKgvmU'}}}, {'AttemptCount': 1}, {'StepName': 'AccuracyGate',
'StartTime': datetime.datetime(2025, 10, 18, 1, 33, 5, 271000,
tzinfo=tzlocal()), 'EndTime': datetime.datetime(2025, 10, 18, 1, 33,
5, 577000, tzinfo=tzlocal()), 'StepStatus': 'Succeeded', 'Metadata':
{'Condition': {'Outcome': 'True'}}}, {'AttemptCount': 1}, {'StepName':
'EvaluateModel', 'StartTime': datetime.datetime(2025, 10, 18, 1, 33,
4, 55000, tzinfo=tzlocal()), 'EndTime': datetime.datetime(2025, 10,
18, 1, 33, 4, 609000, tzinfo=tzlocal()), 'StepStatus': 'Succeeded',
'CacheHitResult': {'SourcePipelineExecutionArn':
'arn:aws:sagemaker:us-east-1:533266958221:pipeline/apple-ci-model-
registry/execution/44ir52uc6spb'}, 'Metadata': {'ProcessingJob':
{'Arn':
'arn:aws:sagemaker:us-east-1:533266958221:processing-job/pipelines-
44ir52uc6spb-EvaluateModel-tA9sXZ2lkr'}}}, {'AttemptCount': 1},
{'StepName': 'TrainModel', 'StartTime': datetime.datetime(2025, 10,
18, 1, 33, 2, 717000, tzinfo=tzlocal()), 'EndTime':
datetime.datetime(2025, 10, 18, 1, 33, 3, 491000, tzinfo=tzlocal()),
'StepStatus': 'Succeeded', 'CacheHitResult':
{'SourcePipelineExecutionArn': 'arn:aws:sagemaker:us-east-
1:533266958221:pipeline/apple-ci-model-registry/execution/
44ir52uc6spb'}, 'Metadata': {'TrainingJob': {'Arn':
'arn:aws:sagemaker:us-east-1:533266958221:training-job/pipelines-
44ir52uc6spb-TrainModel-SRVhPqZEZg'}}}, {'AttemptCount': 1}]

```

Approve and Deploy

Here let's programmatically set approval to `Approved` and deploy the selected model package to an endpoint (`apple-registry-prod`) for live inference

```
import pprint
sm = boto3.client("sagemaker")
mp = sm.describe_model_package(ModelPackageName="arn:aws:sagemaker:us-east-1:533266958221:model-package/apple-leaf-registry/1")
pprint.pprint({k: mp[k] for k in ["ModelPackageStatus", "ModelApprovalStatus", "ModelMetrics", "CreationTime"]})

{'CreationTime': datetime.datetime(2025, 10, 18, 1, 35, 41, 142000, tzinfo=tzlocal()),
 'ModelApprovalStatus': 'PendingManualApproval',
 'ModelMetrics': {'Bias': {},
                  'Explainability': {},
                  'ModelQuality': {'Statistics': {'ContentType': 'application/json',
                                                  'S3Uri': 's3://sagemaker-us-east-1-533266958221/apple-ci-model-registry/44ir52uc6spb/EvaluateModel/output/metrics'}}}},
 'ModelPackageStatus': 'Completed'}
```

Inspect Executions and Artifacts

List pipeline execution steps, fetch statuses and console links for training, processing, and registration jobs, and retrieve the created model package details for traceability.

```
sm.update_model_package(
    ModelPackageArn="arn:aws:sagemaker:us-east-1:533266958221:model-package/apple-leaf-registry/1",
    ModelApprovalStatus="Approved",
    ApprovalDescription="Meets accuracy gate; approving from CI pipeline."
)

{'ModelPackageArn': 'arn:aws:sagemaker:us-east-1:533266958221:model-package/apple-leaf-registry/1',
 'ResponseMetadata': {'RequestId': 'be6714ea-6b31-467c-a2d6-8a26f4ca59c5',
                      'HTTPStatusCode': 200,
                      'HTTPHeaders': {'x-amzn-requestid': 'be6714ea-6b31-467c-a2d6-8a26f4ca59c5',
                                       'strict-transport-security': 'max-age=47304000; includeSubDomains',
                                       'x-frame-options': 'DENY',
                                       'content-security-policy': "frame-ancestors 'none'",
                                       'cache-control': 'no-cache, no-store, must-revalidate',
```

```

    'x-content-type-options': 'nosniff',
    'content-type': 'application/x-amz-json-1.1',
    'content-length': '98',
    'date': 'Sat, 18 Oct 2025 01:43:09 GMT'},
    'RetryAttempts': 0}}

from sagemaker import Session
from sagemaker.model import ModelPackage
sess = Session()

mp_model = ModelPackage(
    role=sagemaker.get_execution_role(),
    model_package_arn="arn:aws:sagemaker:us-east-1:533266958221:model-
package/apple-leaf-registry/1",
    sagemaker_session=sess,
)
predictor = mp_model.deploy(
    endpoint_name="apple-registry-prod",
    initial_instance_count=1,
    instance_type="ml.m5.large"
)

```

```

INFO:sagemaker:Creating model with name: apple-leaf-registry-2025-10-
18-01-43-26-521

```

```

INFO:sagemaker:Creating endpoint-config with name apple-registry-prod
INFO:sagemaker:Creating endpoint with name apple-registry-prod

```

```

-----!

```

```

import pandas as pd
sm = boto3.client("sagemaker")

exec_arn = "arn:aws:sagemaker:us-east-1:533266958221:pipeline/apple-
ci-model-registry/execution/7piguyat6xjf"
steps =
sm.list_pipeline_execution_steps(PipelineExecutionArn=exec_arn)
["PipelineExecutionSteps"]

rows = []
for st in steps:
    rows.append({
        "StepName": st["StepName"],
        "Type": next(iter(st.get("Metadata", {}).keys()), "-"),
        "Status": st["StepStatus"],
        "FailureReason": st.get("FailureReason", ""),
    })
pd.DataFrame(rows)

```

	StepName	Type	Status	FailureReason
0	RegisterModel-RegisterModel	RegisterModel	Succeeded	
1	RegisterModel-RepackModel	TrainingJob	Succeeded	

2	AccuracyGate	Condition	Succeeded
3	EvaluateModel	ProcessingJob	Succeeded
4	TrainModel	TrainingJob	Succeeded

Export Pipeline Definition

Finally, let's save the compiled pipeline JSON definition to version the workflow itself and **enable reproducible CI/CD**.

```
# Save pipeline definition to a file:
with open("pipeline_definition.json", "w") as f:
    f.write(pipeline.definition())
```

INFO:sagemaker.image_uris:image_uri is not presented, retrieving image_uri based on instance_type, framework etc.

WARNING:sagemaker.workflow.utilities:Popping out 'TrainingJobName' from the pipeline definition by default since it will be overridden at pipeline execution time. Please utilize the PipelineDefinitionConfig to persist this field in the pipeline definition if desired.

WARNING:sagemaker.workflow.utilities:Popping out 'ProcessingJobName' from the pipeline definition by default since it will be overridden at pipeline execution time. Please utilize the PipelineDefinitionConfig to persist this field in the pipeline definition if desired.

WARNING:sagemaker.workflow.utilities:Popping out 'TrainingJobName' from the pipeline definition by default since it will be overridden at pipeline execution time. Please utilize the PipelineDefinitionConfig to persist this field in the pipeline definition if desired.

WARNING:sagemaker.estimator:No finished training job found associated with this estimator. Please make sure this estimator is only used for building workflow config

WARNING:sagemaker.workflow._utils:Popping out 'CertifyForMarketplace' from the pipeline definition since it will be overridden in pipeline execution time.

WARNING:sagemaker.workflow.utilities:Popping out 'ModelPackageName' from the pipeline definition by default since it will be overridden at pipeline execution time. Please utilize the PipelineDefinitionConfig to persist this field in the pipeline definition if desired.